

UNSW, School of Mathematics and Statistics

Profs Josef Dick and Kassem Mustapha

MATH3311/MATH5335

Topic 02 – Size, Equality, Norms, Sensitivity

- 1 Magnitude of real numbers
 - Absolute values
- 2 Vector and matrix in MATLAB-Python
- 3 Vector Norms
 - Vector p -norms
 - Equality of vectors
- 4 Errors in vectors
 - Matrix Norms
 - Matrix p -norms
 - Equality of matrices
 - Condition numbers
- 5 Systems of linear equations
 - Sensitivity of linear systems

Absolute values

The magnitude of a real number $\alpha \in \mathbb{R}$ is

$$|\alpha| = \begin{cases} \alpha & \text{if } \alpha \geq 0; \\ -\alpha & \text{if } \alpha < 0. \end{cases}$$

- $|\alpha| = \max(\alpha, -\alpha)$, is continuous, but **not differentiable** at $\alpha = 0$
- $\alpha, \beta \in \mathbb{R}$: $\alpha = \beta \iff |\alpha - \beta| = 0$
- MATLAB/Python: `==` equal, `~/!=` not equal, `abs` absolute value

Example (Computer arithmetic is not associative)

Are α and β equal, where

$$\alpha = \left(\frac{3}{4}\right) \times \left(\frac{1}{5}\right), \quad \beta = \frac{3}{20}$$

- Using exact arithmetic?
- Using MATLAB (see M-file `eqex.m`), Python (`eqex.py`)

Scalars are fine. What about vectors or matrices?

MATLAB file `mateqK.mat` contains two matrices K_1, K_2 . Are they equal?

Vectors and matrices in MATLAB and Python

- Python code

```
# Import the NumPy library
import numpy
# Define a row vector
ar = numpy.array([1, 2, 3])
# Define a column vector
ac = numpy.array([[1], [2], [3]])
# Define a matrix
A = numpy.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

- MATLAB code

```
% Define a row vector
ar = [1 2 3]
% Define a column vector
ac = [1; 2; 3]
% Define a matrix
A = [1 2 3; 4 5 6; 7 8 9]
```

Vector norms

Measuring the magnitude of a vector quantity

- What do we mean by a small or large vector?
- For $\mathbf{x} \in \mathbb{R}^n$ (a **column** vector), we usually measure the size of $\mathbf{x}$ using a vector norm, denoted by $\|\mathbf{x}\|$. There are different norms.
- Recall that, the length of a vector $\mathbf{x} \in \mathbb{R}^n$ is $|\mathbf{x}| = \sqrt{\sum_{i=1}^n x_i^2}$.

Definition (Vector norm)

A **Vector norm** on $\mathbb{R}^n$ is a function $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ satisfying

- 1 $\|\mathbf{x}\| \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$ and $\|\mathbf{x}\| = 0$ iff $\mathbf{x} = \mathbf{0}$
- 2 $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ (triangle inequality)
- 3 $\|\alpha \mathbf{x}\| = |\alpha| \|\mathbf{x}\|$ for all $\alpha \in \mathbb{R}, \mathbf{x} \in \mathbb{R}^n$

- For $\mathbf{x} \in \mathbb{R}^n$ and $p \geq 1$, the p -norms are defined by

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}$$

Vector p -norms

In practice, the most important norms are:

- 1-norm

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$$

- 2-norm

$$\|\mathbf{x}\|_2 = \left(\sum_{i=1}^n |x_i|^2 \right)^{\frac{1}{2}} = |\mathbf{x}|$$

- ∞ -norm or maximum norm

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

- $\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^T \mathbf{x}}$ (dot product $\mathbf{x} \cdot \mathbf{y} = \mathbf{x}^T \mathbf{y} = \sum_{i=1}^n x_i y_i$)

- MATLAB [norm](#), see [vecnorms.m](#); Python [numpy.linalg.norm](#), see [vecnorms.py](#)

- 2-norm: `norm(x,2)` ([MATLAB](#) and [Python](#))
- 1-norm: `norm(x,1)`
- ∞ -norm: `norm(x,inf)`

Vector norms – examples

Example (Vector norms)

Find the 1, 2 and ∞ -norms of the vector

$$\mathbf{v} = (-1, 2, -3, 2)^T$$

Solution

- 1-norm:

$$\|\mathbf{v}\|_1 = \sum_{i=1}^4 |v_i| = 1 + 2 + 3 + 2 = 8.$$

- 2-norm:

$$\|\mathbf{v}\|_2^2 = \sum_{i=1}^4 |v_i|^2 = (-1)^2 + 2^2 + (-3)^2 + 2^2 = 18 \implies \|\mathbf{v}\|_2 = \sqrt{18}.$$

- ∞ -norm:

$$\|\mathbf{v}\|_\infty = \max_{i=1,2,3,4} |v_i| = \max\{1, 2, 3, 2\} = 3.$$

Equality of vectors

Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$. Then,

- $\mathbf{x} = \mathbf{y} \iff x_i = y_i \text{ for all } i = 1, \dots, n$
- $\mathbf{x} = \mathbf{y} \iff \|\mathbf{x} - \mathbf{y}\| = 0$
- MATLAB: `x == y` is a logical array; Python: `x == y` is a Boolean array.
- Rounding error: equal if $\|\mathbf{x} - \mathbf{y}\| \leq \tau(\epsilon, n, \|\mathbf{x}\|, \|\mathbf{y}\|)$

Example (Equality of vectors)

- Are $\mathbf{x} = (0.75 \times 0.2, \frac{3}{4} \times \frac{1}{5}, 0.3)^T$ and $\mathbf{y} = \frac{3}{20}(1, 1, 2)^T$ equal?

MATLAB: M-file `eqvex.m`; Python `eqvex.py`

Warning:

- MATLAB command `length` does **not** give $|\mathbf{x}|$
- Compare MATLAB commands `size`, `length`, `numel`; Python `len`

Errors in vectors

If $\bar{\mathbf{x}} \in \mathbb{R}^n$ is a computed approximation to $\mathbf{x} \in \mathbb{R}^n$, then

- The absolute error is $e_a = \|\bar{\mathbf{x}} - \mathbf{x}\|$.
- The relative error (for $\mathbf{x} \neq \mathbf{0}$) is $e_r = \|\bar{\mathbf{x}} - \mathbf{x}\|/\|\mathbf{x}\|$.
- The largest component of $\mathbf{x}$ has k significant figures if

$$\frac{\|\bar{\mathbf{x}} - \mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} < 0.5 \times 10^{-k}.$$

Thus, the number of significant figures can be estimated as

$$k = \left\lfloor -\log_{10} \left(2 \frac{\|\bar{\mathbf{x}} - \mathbf{x}\|_\infty}{\|\mathbf{x}\|_\infty} \right) \right\rfloor.$$

Errors in vectors – an example

Example

If $\mathbf{x} = (-3.641, 0.7843)^T$ and $\bar{\mathbf{x}} = (-3.633, 0.7915)^T$, then find the absolute and relative errors using the ∞ -norm, and the estimate number of significant figures.

Solution

- $\mathbf{x} - \bar{\mathbf{x}} = (-0.0080, -0.0072)^T$.
- The absolute error is $e_a = \|\mathbf{x} - \bar{\mathbf{x}}\|_\infty = 0.008$.
- The relative error is
$$e_r = \|\mathbf{x} - \bar{\mathbf{x}}\|_\infty / \|\mathbf{x}\|_\infty = 0.008 / 3.641 = 0.0022 < 0.5 \times 10^{-2}.$$
- $-\log_{10}(2e_r) \approx 2.36$, and so, the number of significant figures $k = 2$.

Matrix norms

Definition (Matrix norm)

A **Matrix norm** is a scalar function $\|\cdot\| : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$ satisfying

- ① $\|A\| \geq 0$ for all $A \in \mathbb{R}^{m \times n}$ and $\|A\| = 0 \iff A = \mathbf{0}$.
- ② $\|A + B\| \leq \|A\| + \|B\|$ for all $A, B \in \mathbb{R}^{m \times n}$ (triangle inequality).
- ③ $\|\alpha A\| = |\alpha| \|A\|$ for all $\alpha \in \mathbb{R}$ and $A \in \mathbb{R}^{m \times n}$.

Definition (Consistent matrix norms)

Matrix norms are **consistent** if

$$\|AB\| \leq \|A\| \|B\| \text{ for all } A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{n \times \ell}.$$

Subordinate matrix norm

- **Subordinate matrix p -norm** is defined in terms of vector norms as:

$$\|A\|_p = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_p}{\|\mathbf{x}\|_p}, \quad \text{for } p = 1, 2, \dots$$

- From the above definition, if $A \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{x} \neq \mathbf{0}$, then

$$\|A\mathbf{x}\|_p = \frac{\|A\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \|\mathbf{x}\|_p \leq \|A\|_p \|\mathbf{x}\|_p.$$

- **Frobenius** matrix norm is defined as:

$$\|A\|_F = \left(\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2 \right)^{\frac{1}{2}}, \quad \text{which is Not the same as the } \|A\|_2.$$

- Common subordinate matrix p -norms: $p = 1, 2$ and ∞
 - MATLAB function `norm`; Python `numpy.linalg.norm`
 - 2-norm: `norm(A, 2)`
 - 1-norm: `norm(A, 1)`, ∞ -norm: `norm(A, inf)`
 - Frobenius norm: `norm(A, 'fro')`

Matrix 1-norm

The matrix 1-norm is the maximum absolute column sum, that is, for

$$A = \begin{bmatrix} a_{11} & \dots & a_{1j} & \dots & a_{1n} \\ \vdots & & \vdots & & \vdots \\ \vdots & & \vdots & & \vdots \\ a_{m1} & \dots & a_{mj} & \dots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n},$$

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|.$$

To show this, we need to prove that

$$\|A\|_1 \leq \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}| \quad \text{and} \quad \|A\|_1 \geq \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|.$$

We skip the proof of the first inequality, and show the second one next.

Matrix 1-norm (continued)

For $1 \leq j \leq n$, let $\mathbf{e}_j \in \mathbb{R}^n$ be a column vector where the j^{th} entry is 1 and the rest are zeros (j^{th} canonical basis vector in $\mathbb{R}^n$). Let $\mathbf{c}_j$ be the j^{th} column of A .

$$A\mathbf{e}_j = \begin{bmatrix} a_{11} & \dots & \mathbf{a_{1j}} & \dots & a_{1n} \\ \vdots & & \vdots & & \vdots \\ \vdots & & \vdots & & \vdots \\ a_{m1} & \dots & \mathbf{a_{mj}} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \end{bmatrix} = \begin{bmatrix} a_{1j} \\ a_{2j} \\ a_{3j} \\ \vdots \\ a_{mj} \end{bmatrix} = \mathbf{c}_j.$$

So, for $\mathbf{x} \in \mathbb{R}^n$,

$$\|A\|_1 = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_1}{\|\mathbf{x}\|_1} \geq \max_{1 \leq j \leq n} \frac{\|A\mathbf{e}_j\|_1}{\|\mathbf{e}_j\|_1} = \max_{1 \leq j \leq n} \|\mathbf{c}_j\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|.$$

Matrix ∞ -norm

The matrix ∞ -norm is the maximum absolute row sum, that is, with

$$A = \begin{bmatrix} a_{11} & \dots & a_{1j} & \dots & a_{1n} \\ \vdots & & \vdots & & \vdots \\ a_{\ell 1} & \dots & a_{\ell j} & \dots & a_{\ell n} \\ \vdots & & \vdots & & \vdots \\ a_{m1} & \dots & a_{mj} & \dots & a_{mn} \end{bmatrix},$$

$$\|A\|_{\infty} = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|.$$

Here, we need to show that

$$\|A\|_{\infty} \leq \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}| \quad \text{and} \quad \|A\|_{\infty} \geq \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|.$$

As before, we skip the proof of the first inequality, and show the second one next.

Matrix ∞ -norm (continued)

Suppose that the ℓ^{th} row of A gives the maximum absolute row sum for some $1 \leq \ell \leq m$. That is, $\sum_{j=1}^n |a_{\ell j}| = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$.

Let $\mathbf{v} = [v_j] \in \mathbb{R}^n$ be a column vector such that $|v_j| = 1$ and $v_j a_{\ell j} = |a_{\ell j}|$ for $1 \leq j \leq n$. Then,

$$A\mathbf{v} = \begin{bmatrix} a_{11} & \dots & a_{1j} & \dots & a_{1n} \\ \vdots & & \vdots & & \vdots \\ a_{\ell 1} & \dots & a_{\ell j} & \dots & a_{\ell n} \\ \vdots & & \vdots & & \vdots \\ a_{m1} & \dots & a_{mj} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_j \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} \sum_{j=1}^n a_{1j} v_j \\ \vdots \\ \sum_{j=1}^n |a_{\ell j}| \\ \vdots \\ \sum_{j=1}^n a_{mj} v_j \end{bmatrix}.$$

So, for $\mathbf{x} \in \mathbb{R}^n$,

$$\|A\|_{\infty} = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_{\infty}}{\|\mathbf{x}\|_{\infty}} \geq \frac{\|A\mathbf{v}\|_{\infty}}{\|\mathbf{v}\|_{\infty}} = \sum_{j=1}^n |a_{\ell j}| = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|.$$

Matrix 2-norm, $A \in \mathbb{R}^{m \times n}$

Let $\lambda_1, \dots, \lambda_n$ (**all** ≥ 0) be the eigenvalues of $B := A^T A$. Then

$$\|A\|_2 = \sqrt{\lambda}, \quad \text{where } \lambda = \max_{1 \leq i \leq n} \lambda_i.$$

Here, we need to show that $\|A\|_2 \leq \sqrt{\lambda}$ (using the eigen-decomposition of B) and that $\|A\|_2 \geq \sqrt{\lambda}$. **Let us show the second inequality.**

By **definition**, $B\mathbf{v} = \lambda\mathbf{v}$ for some non-zero vector $\mathbf{v} \in \mathbb{R}^n$. Then,

$$\|A\mathbf{v}\|_2^2 = (A\mathbf{v})^T (A\mathbf{v}) = \mathbf{v}^T A^T A \mathbf{v} = \mathbf{v}^T B \mathbf{v} = \lambda \mathbf{v}^T \mathbf{v} = \lambda \|\mathbf{v}\|_2^2.$$

Thus, for $\mathbf{x} \in \mathbb{R}^n$,

$$\|A\|_2 = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\mathbf{x}\|_2}{\|\mathbf{x}\|_2} \geq \frac{\|A\mathbf{v}\|_2}{\|\mathbf{v}\|_2} = \sqrt{\lambda}.$$

Check **eig** in MATLAB, **numpy.linalg.eig** and **scipy.linalg.eigh** in Python.

Matrix norms – examples

Example (Matrix norms)

Find the 1, 2, infinity and Frobenius norms of

$$A = \begin{pmatrix} 1 & -3 & 2 \\ -2 & 2 & 4 \end{pmatrix}.$$

Solution

- $\|A\|_1 = \max\{3, 5, 6\} = 6$, $\|A\|_\infty = \max\{6, 8\} = 8$.
- $\|A\|_F = \sqrt{1^2 + (-3)^2 + 2^2 + (-2)^2 + 2^2 + 4^2} = \sqrt{38}$.
- $\|A\|_2 = ??$

$$B = A^T A = \begin{pmatrix} 5 & -7 & -6 \\ -7 & 13 & 2 \\ -6 & 2 & 20 \end{pmatrix}.$$

Eigenvalues of B are 0, 14, 24. Then $\|A\|_2 = \sqrt{24}$.

Subordinate matrix p -norms are consistent

- For any matrix $A \in \mathbb{R}^{m \times n}$ and any matrix $B \in \mathbb{R}^{n \times \ell}$,

$$\|AB\|_p \leq \|A\|_p \|B\|_p, \quad \text{for } p \geq 1.$$

Proof. For $\mathbf{x} \in \mathbb{R}^\ell$,

$$\begin{aligned} \|AB\|_p &= \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|AB\mathbf{x}\|_p}{\|\mathbf{x}\|_p} = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A(B\mathbf{x})\|_p}{\|\mathbf{x}\|_p} \\ &\leq \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\|_p \|B\mathbf{x}\|_p}{\|\mathbf{x}\|_p} \\ &\leq \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|A\|_p \|B\|_p \|\mathbf{x}\|_p}{\|\mathbf{x}\|_p} = \|A\|_p \|B\|_p. \end{aligned}$$

Equality of matrices

Let $A = [a_{ij}]$ and $B = [b_{ij}]$ be two matrices in $\mathbb{R}^{m \times n}$.

- $A = B \iff a_{ij} = b_{ij}$ for all $i = 1, \dots, m, j = 1, \dots, n$.
- $A = B \iff \|A - B\| = 0$.
- MATLAB: `A == B` returns a logical array; Python: `A == B` returns a Boolean array.
- A and B are considered equal (up to rounding errors) if $\|A - B\| \leq \tau(\epsilon, m, n, \|A\|, \|B\|)$, where τ is a tolerance that depends on ϵ , the matrix dimensions m, n , and the norms of A and B .

Example (Equality of matrices)

- Are

$$A = \begin{bmatrix} 1 & 2 \\ -3 & 2 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 2 & 0 \\ -3 & 2 & 0 \end{bmatrix}.$$

equal?

- MATLAB: M-file `eqmex.m`; Python `eqmex.py`

Condition numbers

Definition

For any nonsingular matrix $A \in \mathbb{R}^{n \times n}$, the **condition number** is

$$\kappa(A) = \|A\| \|A^{-1}\|, \quad \text{where } \|\cdot\| \text{ is any matrix norm.}$$

- For a consistent matrix norm, with $\|I_n\| = 1$,

$$I_n = AA^{-1} \Rightarrow 1 = \|I_n\| = \|AA^{-1}\| \leq \|A\| \|A^{-1}\| \Rightarrow \kappa(A) \geq 1.$$

- $\kappa(\alpha I_n) = 1$, for $\alpha \in \mathbb{R}$, $\alpha \neq 0$.
- For any **real symmetric nonsingular matrix** A with eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$, using the 2-norm,

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2 = \frac{\max_{1 \leq i \leq n} |\lambda_i|}{\min_{1 \leq i \leq n} |\lambda_i|}.$$

- Note:** A larger $\kappa(A)$ means the solution $\mathbf{x}$ of $A\mathbf{x} = \mathbf{b}$ is more sensitive to small errors in the data, making the problem *ill-conditioned*. **More details are provided in the coming slides.**

Ill-conditioned matrices

- A is **ill-conditioned** if $\kappa(A)$ is **large**
 - What count as large? A common rule is when $\kappa(A) > 1/\epsilon \approx 10^{16}$ where ϵ is the relative machine precision
- Reciprocal condition number:

$$0 < \text{rcond}(A) = \frac{1}{\kappa(A)} \leq 1$$

- Well conditioned $\iff \text{rcond}(A)$ is close to 1.
 - Badly conditioned $\iff \text{rcond}(A)$ is close to 0.
- MATLAB functions, Python functions
 - $\text{cond}(A, p)$, [numpy.linalg.cond\(A, p\)](#) use the p -norm for $p = 1, 2, \infty$.
 - $\text{rcond}(A)$ gives an estimate of $1/\kappa(A)$ using the 1-norm.

Systems of linear equations

- System $A\mathbf{x} = \mathbf{b}$, $A \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$

$$\begin{array}{ccccccc}
 a_{11}x_1 & + & a_{12}x_2 & + & \cdots & + & a_{1n}x_n & = & b_1 \\
 a_{21}x_1 & + & a_{22}x_2 & + & \cdots & + & a_{2n}x_n & = & b_2 \\
 \vdots & & \vdots & & & & \vdots & & \vdots \\
 a_{m1}x_1 & + & a_{m2}x_2 & + & \cdots & + & a_{mn}x_n & = & b_m
 \end{array}$$

- m linear equations in n variables.
 - Each row represents one equation.
 - All m equations must be satisfied simultaneously.
- Exploit the structure of A to solve the system in ways that are
 - numerically stable** – limiting the effects of errors in the data
 - efficient** – in time (flops) and memory
- Systems are
 - Well-determined**: $m = n$, same number of equations and variables
 - Over-determined**: $m > n$, more equations than variables
 - Under-determined**: $m < n$, fewer equations than variables

Sensitivity of linear systems

- Let A be a nonsingular matrix. How do errors in the data A and $\mathbf{b}$ affect the computed solution of the linear system $A\mathbf{x} = \mathbf{b}$?
- Perturbed system

$$(A + \delta_A)(\mathbf{x} + \delta_{\mathbf{x}}) = \mathbf{b} + \delta_{\mathbf{b}},$$

where $A + \delta_A$ is assumed to be nonsingular.

Thus, $\delta_{\mathbf{x}}$ is the change in the solution $\mathbf{x}$ due to changes δ_A and $\delta_{\mathbf{b}}$ in the matrix A and the vector $\mathbf{b}$, respectively.

- Relative error in input data are

$$\rho_A = \frac{\|(A + \delta_A) - A\|}{\|A\|} = \frac{\|\delta_A\|}{\|A\|} \quad \text{and} \quad \rho_{\mathbf{b}} = \frac{\|\delta_{\mathbf{b}}\|}{\|\mathbf{b}\|}.$$

What is the relative error in the solution ($\rho_{\mathbf{x}}$)? To discuss this, suppose that $\|(A + \delta_A)^{-1}\| \leq \alpha \|A^{-1}\|$ for some constant $\alpha > 1$.

Sensitivity of linear systems (continued)

From $A\mathbf{x} = \mathbf{b}$ and $(A + \delta_A)(\mathbf{x} + \delta_{\mathbf{x}}) = \mathbf{b} + \delta_{\mathbf{b}}$, we have

$$(A + \delta_A)\delta_{\mathbf{x}} = \mathbf{b} + \delta_{\mathbf{b}} - (A + \delta_A)\mathbf{x} = \delta_{\mathbf{b}} - \delta_A \mathbf{x}.$$

Thus, $\delta_{\mathbf{x}} = (A + \delta_A)^{-1}(\delta_{\mathbf{b}} - \delta_A \mathbf{x})$, and hence,

$$\begin{aligned}\|\delta_{\mathbf{x}}\| &= \|(A + \delta_A)^{-1}(\delta_{\mathbf{b}} - \delta_A \mathbf{x})\| \leq \|(A + \delta_A)^{-1}\| (\|\delta_{\mathbf{b}}\| + \|\delta_A\| \|\mathbf{x}\|) \\ &\leq \alpha \|A^{-1}\| (\rho_{\mathbf{b}} \|\mathbf{b}\| + \rho_A \|A\| \|\mathbf{x}\|).\end{aligned}$$

Since $\|\mathbf{b}\| = \|A\mathbf{x}\| \leq \|A\| \|\mathbf{x}\|$, it follows that

$$\|\delta_{\mathbf{x}}\| \leq \alpha \|A^{-1}\| \|A\| \|\mathbf{x}\| (\rho_{\mathbf{b}} + \rho_A).$$

Therefore, the relative error in the solution is

$$\rho_{\mathbf{x}} = \frac{\|(\mathbf{x} + \delta_{\mathbf{x}}) - \mathbf{x}\|}{\|\mathbf{x}\|} = \frac{\|\delta_{\mathbf{x}}\|}{\|\mathbf{x}\|} \leq \alpha \kappa(A) [\rho_{\mathbf{b}} + \rho_A].$$

Ignoring α , the relative error in the solution is bounded by $\kappa(A)$ times the relative error in the data.

Accuracy of computed solution – example

Example (Accuracy of the computed solution)

A real symmetric matrix A is known **exactly** and has ordered eigenvalues

$$1004.2, \quad 999.8, \quad \dots, \quad 0.0034, \quad 0.0005,$$

while the right-hand-side vector $\mathbf{b}$ is measured only to an accuracy of 6 significant figures.

- 1 Explain why A is nonsingular.
- 2 Estimate the condition number of A .
- 3 What is the relative error in the inputs A and $\mathbf{b}$.
- 4 Estimate the relative error of the computed solution to $A\mathbf{x} = \mathbf{b}$.
- 5 What confidence do you have in the computed solution?

Accuracy of computed solution – example

Solution (Accuracy of computed solution)

- ① *A is a real symmetric matrix and its smallest eigenvalue is $0.0005 > 0$, so A is nonsingular.*
- ② *$\kappa_2(A) = |\lambda_{\max}|/|\lambda_{\min}| = 1004.2/0.0005 \approx 2 \times 10^6$.*
- ③ *Relative errors in the inputs:*
 - *A is known **exactly**, so the relative error in A is $\rho_A = \epsilon$*
 - ***b** is known to 6 significant figures, so the relative error in **b** satisfies $\rho_b \leq 0.5 \times 10^{-6}$.*

- ④ *Estimation of the relative error in the computed solution:*

$$\kappa_2(A) (\rho_A + \rho_b) \approx 2 \times 10^6 (2 \times 10^{-16} + 0.5 \times 10^{-6}) \approx 1.$$

- ⑤ *A relative error of order 1 means that the computed solution may have essentially no correct significant digits, so there is **no confidence** in the computed solution.*

Conditioning of linear systems

Definition (Well-conditioned)

Let $A \in \mathbb{R}^{n \times n}$ be a nonsingular matrix. The linear system $A\mathbf{x} = \mathbf{b}$ is **well-conditioned** if a small change in A or $\mathbf{b}$ leads to only a small change in $\mathbf{x}$.

- The linear system $A\mathbf{x} = \mathbf{b}$ is **ill-conditioned** (or *badly conditioned*) if a small change in A or $\mathbf{b}$ can result in a large change in $\mathbf{x}$.
 - Then we expect trouble when computing $\mathbf{x}$ numerically.
- The matrix A is **well-conditioned** (respectively, **ill-conditioned**) if the linear system $A\mathbf{x} = \mathbf{b}$ is **well-conditioned** for every $\mathbf{b}$ (respectively, **ill-conditioned** for some $\mathbf{b}$).
- The Hilbert matrix $H = [h_{ij}]$, where $h_{ij} = \frac{1}{i+j-1}$, is an example of an ill-conditioned matrix.

See the MATLAB-Python files: **hilbex.m** and **hilbex.py**